ECG Simulator Documentation
Part 6: System Architecture & Design

Module Organization, Class Hierarchy, Extension Points

June 2026

Contents

1 Three-Layer Architecture

1.1 Overview

The ECG simulator is organized into three distinct layers:

1. **Physics Layer**: Cardiac physiology (cell models, propagation, dipoles)
2. **Simulation Layer**: Algorithm selection (Phase 1/2 vs Phase 3), beat management
3. **GUI Layer**: PyQt6 interface, real-time visualization, parameter control

1.2 Layer Interaction

```
GUI Layer (main_window.py, parameter_panel.py)
    |
    | controls
    |
Simulation Layer (graph_engine.py, cable_engine.py)
    |
    | uses
    |
Physics Layer (cell_models/, tissue/, ecg/)
```

1.3 Data Flow

1. User adjusts HR or parameters in GUI
2. GUI updates ParameterModel (single source of truth)
3. SimulationEngine queries ParameterModel for each beat
4. Engine executes propagation algorithm
5. Forward model computes 12-lead ECG voltages
6. GUI displays voltages on pyqtgraph widget

2 Physics Layer (Core Modules)

2.1 Module Organization

Module	Purpose	Lines
cardiac_sim/core/cell_models/fitzhugh_nagumo.py	FHN cell model ODEs	150
cardiac_sim/core/tissue/cable_1d.py	1-D monodomain integration	500
cardiac_sim/core/tissue/graph.py	DAG + BFS conduction	350
cardiac_sim/core/ecg/lead_field.py	12-lead lead vectors	150

2.2 Class Hierarchy: Cell Models

```
BaseCellModel (abstract)
     FitzHughNagumoCell
     [Future: HodgkinHuxley, Courtemanche, tenTusscher]

Key Methods:
  - get_initial_state()
```

```
  - compute_derivatives(t, state)
  - stimulate(t_start, duration, amplitude)
  - v_to_mv(v_norm, v_rest, v_peak)
```

2.2.1 FitzHughNagumoCell Interface

```
blueclass FitzHughNagumoCell(BaseCellModel):
    bluedef get_initial_state(self) -> np.ndarray:
gray         gray#gray grayReturnsgray gray[grayV_RESTgray,gray grayW_RESTgray]
        bluereturn np.array([-1.200, -0.625])

    bluedef compute_derivatives(self, t, state):
gray         gray#gray graySolvesgray graydimensionlessgray grayFHNgray
             grayODEs
gray         gray#gray grayScalesgray graytogray grayphysicalgray graytimegray
             grayviagray grayTAU_S
        v, w = state
        dv = v - v**3/3 - w + I_ext
        dw = self.eps * (v + self.a - self.b * w)
        bluereturn np.array([dv, dw])

    bluedef v_to_mv(self, v_norm, v_rest, v_peak):
gray         gray#gray grayMapsgray gray[-1.2,gray gray2.0]gray gray   gray
             gray[-85,gray gray+30]gray graymV
        bluereturn v_rest + (v_peak - v_rest) * \
                (v_norm - self.V_REST) / (self.V_PEAK - self.V_REST)
```

2.3 Cable Model (1-D Tissue)

```
blueclass Cable1D:
    bluedef __init__(self, params):
gray         gray#gray grayInitializegray grayVgray,gray grayWgray graystate
             gray grayarraysgray gray(18gray graycellsgray)
        self.V = np.ones(18) * self.V_REST
        self.W = np.ones(18) * self.W_REST
        self._check_cfl()

    bluedef new_beat(self, sa_fire_time, params):
gray         gray#gray grayResetgray graystategray,gray graymarkgray graySAgray
             grayactivated
        self.V.fill(self.V_REST)
        self.W.fill(self.W_REST)
        self._active_beat_start = sa_fire_time

    bluedef advance(self, dt):
gray         gray#gray grayIntegrategray graydtgray graysecondsgray grayviagray
             grayn_microgray graysteps
gray         gray#gray grayReturnsgray graydictgray grayofgray graynewlygray
             grayactivatedgray graynodes
        bluefor _ bluein bluerange(n_micro):
            self._step_atrial(DT_INT)
            self._step_ventricular(DT_INT)
        bluereturn newly_activated_dict
```

3 Simulation Layer

3.1 SimulationEngine Interface

```
SimulationEngine (abstract)
     GraphEngine (Phase 1/2: BFS-based)
     CableEngine (Phase 3: FHN cable)

Key Methods:
  - generate_beat(hr, params) -> dict[node -> t_act]
  - get_activation_times() -> dict
  - advance_time(dt)
```

3.2 GraphEngine (Phase 1/2)

```
BFS Algorithm:
1. Start queue with SA node (t=0)
2. For each node:
   a. Compute conduction time to each neighbor
   b. Check if neighbor is refractory
   c. If NOT refractory: add to queue with new activation time
   d. If refractory: skip
3. Return dict[node -> activation_time] for entire beat

Refractory Check:
  t_candidate = t_current + conduction_time
  if t_candidate < last_activation[node] + ERP[node]:
      # Skip (still refractory)
  else:
      # Conduct
```

3.2.1 Key Implementation Details

```
bluedef run_bfs(self, sa_fire_time, params):
    activation_times = {}
    queue = deque([(SA_NODE, sa_fire_time)])

    bluewhile queue:
        current_node, t_curr = queue.popleft()
        activation_times[current_node] = t_curr

gray        gray#gray grayFindgray grayrefractorygray graystatus
        last_act = self._last_activation.get(current_node, -inf)
        blueif t_curr < last_act + ERP[current_node]:
            bluecontinue gray gray#gray graySkipgray grayifgray graystillgray
                grayrefractory

gray        gray#gray grayProcessgray grayneighbors
        bluefor neighbor, cond_time bluein self.adjacency[current_node]:
            t_next = t_curr + cond_time
            queue.append((neighbor, t_next))

    bluereturn activation_times
```

3.3 CableEngine (Phase 3)

```
Phase 3: FHN Cable Integration
1. Initialize 18-cell cable (8 atrial + 10 ventricular)
2. Stimulate SA node at t=sa_fire_time
3. Advance cable.advance(dt) every 33 ms
4. Cable returns dict of newly activated nodes
5. Accumulate activation times incrementally
6. Compute dipole moments from activation_times
7. Project to 12 leads

Architecture:
  - _cable: Cable1D object (manages state V, W)
  - _active_beats: List of BeatRecord objects
  - _current_beat_nodes: Dict tracking activated nodes per beat
  - _nodes: 18 ECGNode objects (anatomical positions)
  - _forward_model: LeadField for projection
```

4 GUI Layer

4.1 PyQt6 Widget Hierarchy

```
MainWindow (QMainWindow)
     ControlPanel (QWidget)
        HR Spinner (QSpinBox)
        Engine Selector (QComboBox)
        Control Buttons (QPushButton)
        Display Label (QLabel)
     ECGDisplay (PlotWidget from pyqtgraph)
        12 curves (one per lead)
     ParameterPanel (QScrollArea)
         SANodeGroup (QGroupBox)
         AVNodeGroup (QGroupBox)
         ... (12 groups total)
         ApplyButton (QPushButton)
```

4.2 Signal/Slot Architecture

```
gray#gray grayUsergray graychangesgray grayHRgray grayspinbox
spinbox.valueChanged.connect(self.on_hr_changed)

bluedef on_hr_changed(self, new_hr):
     self.params.sa.cycle_length = 60 / new_hr
gray     gray#gray grayChangesgray graytakegray grayeffectgray grayongray
         graynextgray graybeat

gray#gray grayTimergray grayupdatesgray grayECGgray graydisplay
self.timer.timeout.connect(self.update_display)

bluedef update_display(self):
gray     gray#gray grayAdvancegray graysimulation
     self.engine.advance(DT_FRAME)

gray     gray#gray grayGetgray grayECGgray grayvoltages
```

```
    ecg_data = self.engine.get_ecg_data()

gray    gray#gray grayUpdategray grayallgray gray12gray graycurves
    bluefor i, lead bluein blueenumerate(self.leads):
        self.plot.getPlotItem().curves[i].setData(ecg_data[i])
```

5 Plugin System

5.1 Plugin Architecture

```
PluginBase (abstract)
    StaticPlugin (cached, whole-beat effect)
    StatefulPlugin (per-beat, temporary override)

Stacking Mechanism:
1. Phase 1/2 BFS algorithm runs
2. For each plugin in stack:
   - Modify activation_times dict or conductances
   - Can disable regions (set conductance → 0)
   - Can prolong refractories
3. Result: Composite arrhythmia
```

5.2 Available Plugins

Plugin	**Mechanism**
AVBlockI	Prolong av_delay
AVBlockII_Mobitz1	Reduce av_conductance gradually
AVBlockII_Mobitz2	Reduce his_conductance abruptly
AVBlockIII	Set av_conductance → 0
LBBB	Set lbb_conductance → 0
RBBB	Set rbb_conductance → 0
LAHB	Set lahb_conductance → 0
LPHB	Set lphb_conductance → 0
AF	Insert fibrillatory waves into atrial rate
PVC	Inject ectopic stimulus at coupling interval
Sinus Bradycardia	Increase sa cycle_length

6 Data Flow Example: Normal Beat

1. User sets HR = 70 bpm
2. GUI calculates cycle_length = 60/70 0.857 s
3. Timer tick at t=0: SA fires
4. BFS/Cable propagates: SA → RA (124 ms) → LA (191 ms) → AV (307 ms) → His (435 ms) → LV (645 ms)
5. Dipole computation for each region
6. Lead field projection: 18-D position vector → 12-lead voltages
7. Display updates all 12 curves
8. Next cycle$_{l}ength : SAfiresagain$

7 Performance Characteristics

7.1 Computational Complexity

Engine	BFS Complexity	Actual Rate	
Phase 1/2	O(V + E) = O(18 + edges)	¿1000 Hz	
Phase 3	O(N_cellsN_steps)	2.2× real-time	

7.2 Memory Usage

- State arrays: 18 cells × 2 variables (V, W) × 8 bytes = 288 bytes
- ECG buffer: 10 seconds × 30 FPS × 12 leads × 8 bytes = 28.8 KB
- Activation times: 18 nodes × 100 beats × 8 bytes = 14.4 KB
- Total: ¡100 MB for typical 1-hour simulation

8 Extension Points

8.1 Adding a New Cell Model

1. Create subclass of BaseCellModel
2. Implement: get_initial_state(), compute_derivatives()
3. Add to cable_1d.py imports
4. Update engine selector to include new model

8.2 Adding a New Plugin

1. Create class inheriting PluginBase
2. Implement: apply() method (modifies activation times or conductances)
3. Register in plugin manager
4. Add to GUI parameter panel

8.3 Adding a New Lead Vector

1. Define unit vector (X, Y, Z) in cardiac coordinate system
2. Add to LEAD_MATRIX in lead_field.py
3. Update lead names list
4. GUI automatically includes in display